



Python Programming Foundations

Week 3 - Organizing Code and Data
Day 1: "Naming Responsibility with Functions"

1



Code Can Work And Still Be Messy

A program can run correctly and still be hard to read, change, or explain.

Week 3 adds a new goal: shaping code so the purpose is easier to see.

Today, watch for:

- ▶ repeated code
- ▶ unclear responsibility
- ▶ logic that deserves a name
- ▶ code that becomes easier to explain after refactoring

2



working but hard to scan

```
def describe_order(items, user):
    # calculate
    subtotal = 0
    for item in items:
        subtotal += item.price * item.qty
    discount = 0
    if user.has_coupon:
        discount = subtotal * 0.10
    tax = (subtotal - discount) * 0.07
    total = subtotal - discount + tax

    # decide
    if total >= 100:
        message = "Great choice! You get free shipping."
    elif total >= 50:
        message = "You're almost there for free shipping!"
    else:
        message = "Add a few more items to save on shipping."

    # show
    print(f"Order summary:")
    print(f"Items: {len(items)}")
    print(f"Subtotal: ${subtotal:.2f}")
    print(f"Discount: ${discount:.2f}")
    print(f"Tax: $${tax:.2f}")
    print(f"Total: ${total:.2f}")
    print(f"{message}")
```

same idea, easier to follow

```
def calculate_subtotal(items, user):
    subtotal = 0
    for item in items:
        subtotal += item.price * item.qty
    discount = 0
    if user.has_coupon:
        discount = subtotal * 0.10
    tax = (subtotal - discount) * 0.07
    total = subtotal - discount + tax

    def show_order_summary(subtotal, discount, tax, total, message):
        print(f"Order summary:")
        print(f"Items: {len(items)}")
        print(f"Subtotal: ${subtotal:.2f}")
        print(f"Discount: ${discount:.2f}")
        print(f"Tax: $${tax:.2f}")
        print(f"Total: ${total:.2f}")
        print(f"{message}")

    def describe_order(items, user):
        subtotal, discount, tax, total = calculate_subtotal(items, user)
        message = show_order_summary(subtotal, discount, tax, total, message)
        show_order_summary(subtotal, discount, tax, total, message)
```

Messy vs. Structured

3



Flow Becomes Shape

Week 2 focused on control flow:

- ▶ conditions choose
- ▶ loops repeat
- ▶ output shows the result

Week 3 asks how that logic should be organized so another person can understand it.

4



Week 2 Concepts Flow Into Functions

Functions organize the skills you already know.

variables

- store values
- keep track
- use in calculations

conditions

- make decisions
- choose a path
- compare values

loops

- repeat steps
- update values
- stop at the right time

function

- named responsibility**
- Does one clear job from start to finish
- Uses variables, conditions, and loops inside
- Has a name so we can use it again

- returns a result
- gives back a value when needed
- reusable building block
- use it in many places without rewriting

Functions organize what you already know into clear, reusable steps. They don't replace conditions or loops — they put them to work.

🧠 Your Week 2 skills. → 🧩 Functions bring them together. → ⭐ Better programs. Easier to build.

"Flow" becomes "Shape"

5

What Counts as Success Today

Repeated or related logic can be named, separated, reused, and explained.

For each function, ask:

What job does it own?	What information does it need?	What result or behavior does it produce?	Where is it called?
-----------------------	--------------------------------	--	---------------------

6

Functions Turn Repetition into Responsibility

The same idea, in four steps.

- 1 Repeated Logic**
The same steps appear in multiple places.
- 2 Named Responsibility**
Give the steps a name and a clear job to do.
- 3 Function Call**
Use the name to run the job when needed.
- 4 Cleaner Program**
Less clutter, easier to read, easier to improve.

Functions don't change your logic — they give it a name and a purpose. Some skills. Better organization.

Today's Success Pattern

7

What We Will Use Today

- ▶ `'def'`
- ▶ function names
- ▶ function calls
- ▶ parameters
- ▶ return values
- ▶ small refactors

These tools help us give code a clear job and reuse that job safely.

8

Functions Toolbox

Simple skills. Powerful results.

Functions help you organize what you already know. One clear job. Use it again and again.

Today's Toolbox

9

What We Will "Skip" For Now

We will skip recursion, lambdas, decorators, and advanced imports for now.

Today's goal is smaller:

- ▶ find repeated or related logic
- ▶ name one clear responsibility
- ▶ write a function
- ▶ call it correctly

10

Parked for Later

Useful topics. Not our focus today.

We'll use these. Just not today. Today we build strong foundations.

recursion	lambdas	decorators	advanced imports
A function calling itself to solve smaller versions of a problem.	Small anonymous functions used for short, simple tasks.	A way to add extra behavior to functions clearly and predictably.	Tools for organizing and controlling how modules and packages load.
We'll revisit this later.	We'll revisit this later.	We'll revisit this later.	We'll revisit this later.

Master the basics first. These topics will make even more sense later.

Parked for Later

11

Review Coding Lab

12

Repeated Code Is A Signal

Repeated code often means the program is doing the same job more than once.

That repeated job may deserve a function.

Look for repeated:

- ▶ calculations
- ▶ output formatting
- ▶ validation checks
- ▶ decision patterns

Spot the Pattern. Simplify the Code.
When the same job shows up more than once, it's a clue to inspect for a possible function.

First time

```
1 # Process scores for math
2 total = 0
3 for score in math_scores:
4     if score >= 68:
5         total = total + score
6     count = count + 1
7 average_math = total / count
```

Second time

```
1 # Process scores for science
2 total = 0
3 for score in science_scores:
4     if score >= 68:
5         total = total + score
6     count = count + 1
7 average_science = total / count
```

same job repeated

These blocks do the same job with different data.
This repetition is a clue to create a function.

Inspect for a function. Name the job once. Use it many times.

Repeated Code is "Sign"

13

14

Demo #1

"Repeated Code Before Functions"

Demo 1: Repeated Code Before Functions

Watch for repeated logic - Before changing code, identify:

- ▶ which lines repeat
- ▶ what job those lines perform
- ▶ what would be annoying to update later

15

16

A Function Owns One Clear Job

A function is a named piece of code with a clear responsibility.

Good function names usually describe the job:

- ▶ `'calculate_total'`
- ▶ `'get_status'`
- ▶ `'show_result'`

If the name is hard to choose... the job may not be clear yet.

One Function. One Job.
Clear input. Clear output. Clear responsibility.

calculate_total

INPUT: amount: 100.00, tax_rate: 0.07

JOB:
calculate total with tax

OUTPUT: total_with_tax: 107.00

This function has one job and does it well.

One function. One responsibility.
Easy to understand. Easy to test. Easy to reuse.

Functions have only ONE job

17

18

Function Shape

Basic function shape:

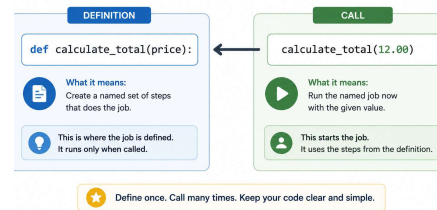
```
1 def function_name(parameter):
2     result = parameter + 1
3     return result
4
```

The definition creates the function. The call runs it.

19

Definition vs Call

Define it once. Call it when you need it.



Function Signature (“Shape”)

20

Demo #2

“Refactor Into a Function”

21

Demo 2: What to Watch For

Watch what changes after the repeated logic becomes a function.

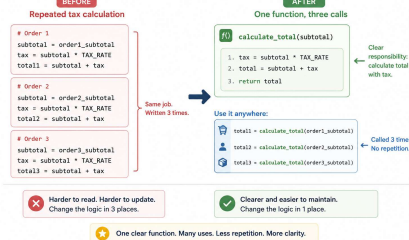
Trace:

- ▶ the function definition
- ▶ the parameter going in
- ▶ the return value coming out
- ▶ the function call being reused

22

From Repetition to Clarity

Same job. Less code. Easier to read.



Refactoring

23

Values Come In, Results Come Out

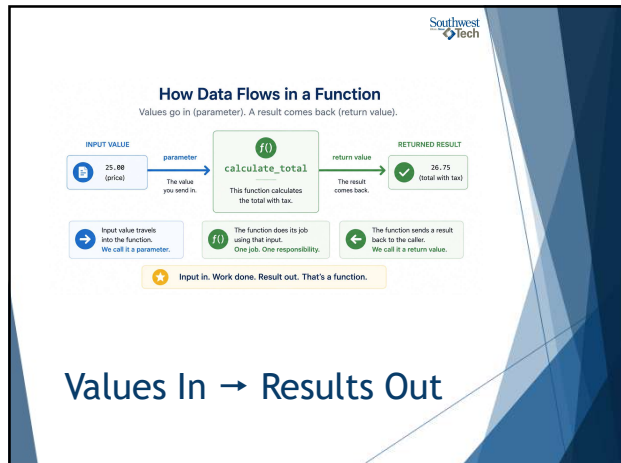
Parameters are information the function receives.

Return values are information the function sends back.

For the beginning mental model:

input value -> function job -> returned result

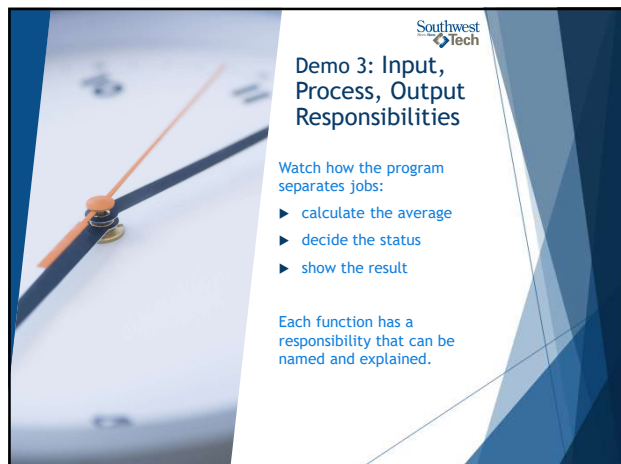
24



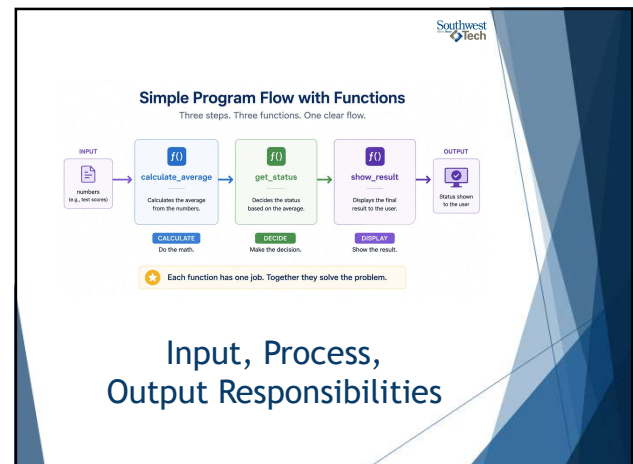
25



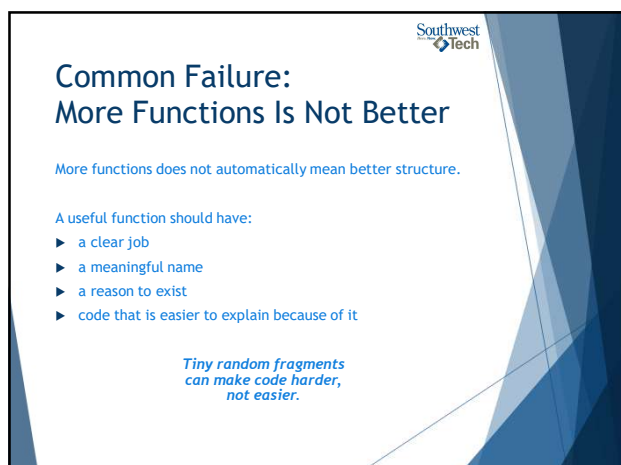
26



27



28



29



30

Common Failure: Vague Function Names

Names should help the reader understand the job.

Weak names:

- ▶ `'do_it'`
- ▶ `'thing'`
- ▶ `'process'`

Clearer names:

- ▶ `'calculate_total'`
- ▶ `'get_status'`
- ▶ `'show_result'`

31

Assignment #4

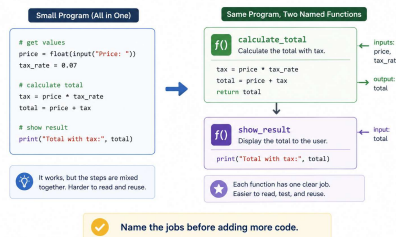
For Assignment 4, build or revise one small program using functions.

A good first version should include:

- ▶ at least two functions
- ▶ meaningful function names
- ▶ correct function calls
- ▶ one parameter or return value if introduced
- ▶ behavior you can explain

32

From Small Program to Named Functions



Assignment #4

33

Function Planning Checklist

Before coding, write a short function plan.

For each function, list:

- ▶ name
- ▶ job
- ▶ information needed
- ▶ result or visible behavior

Then write the smallest version that proves the plan works.

34

Evidence For A Function Program

Your submission should show working code and explainable structure.

Useful evidence includes:

- ▶ one working `.py` file
- ▶ at least two functions
- ▶ readable names
- ▶ visible output or behavior
- ▶ a short explanation of each function's job

35

Function Evidence: Small, Clear, and Testable

```

1 TAX_RATE = 0.07
2
3 def calculate_total(price, rate=TAX_RATE):
4     tax = price * rate
5     total = price + tax
6     return total
7
8 def show_result(total):
9     print("Total with tax: ${total:.2f}")
10
11 # main program
12 price = float(input("Enter price: "))
13 total = calculate_total(price)
14 show_result(total)

```

Sample Output

```

> Enter price: 25.00
Total with tax: $26.75

```

Function Cards

f0 calculate_total(price, rate)
Calculate the total with tax.

Input: price, rate
Output: total

f0 show_result(total)
Display the total to the user.

Input: total
Output: (printed message)

Explanation

The program has two functions:
 • `calculate_total` does the math.
 • `show_result` displays the result.
 The main program gets input, calls the functions, and produces the output.

Evidence

36



37